StudyBuddy – Alexa+ MCP Server: Product Requirements Document

Amazon "Build, Ship, Shape" Hackathon · Alexa+ track · Team of 4 · Version 1.0

> **Note:** This PRD is based on the Alexa+ track text: build a self-hosted MCP server (spec 2025-11-25 or later, Streamable HTTP) or an Agent Skill. Before final submission, re-read the full Devpost rules and judging criteria and adjust any section that conflicts.

1. Overview

Product: StudyBuddy is a voice-first study helper exposed to Alexa+ through an MCP server. A student says things like "Alexa, save a note about deadlocks" or "Alexa, quiz me on my OS notes," and StudyBuddy stores notes, generates quizzes from them, grades spoken answers, and schedules spaced-repetition reviews.

Problem: Students keep notes in many places, rarely test themselves, and forget deadlines. Voice makes quick capture and quizzing possible while walking, cooking, or commuting.

Goal: Ship a demo-ready, working MCP server plus a web chat demo that shows the full flow end to end.

Target users: College students (primary), self-learners preparing for exams.

Success criteria (hackathon):

- MCP server passes the MCP Inspector and connects to a client over Streamable HTTP.
- All five tools work end to end with real data.
- A 2–3 minute demo video shows capture → quiz → grade → review schedule.
- Public HTTPS URL stays up through judging.

2. Scope

In scope (MVP): note capture, quiz generation, answer grading, review scheduling, deadline list, user login, web demo chat.

Out of scope: mobile app, payments, multi-language, OCR of handwritten notes, model fine-tuning.

Stretch: PDF note upload, streak and progress dashboard, Hindi voice support.

3. User Stories

1. As a student, I save a short note by voice so I do not lose an idea.
2. As a student, I ask for a quiz on one subject so I can test myself.
3. As a student, I answer by voice and get a score with a short explanation.
4. As a student, I get told which topics to review today.
5. As a student, I list upcoming deadlines and add new ones.
6. As a user, I log in once and my data stays private to me.

4. Functional Requirements

ID	Requirement	Priority
FR-1	Server implements MCP spec 2025-11-25+ over Streamable HTTP with tool discovery	Must
FR-2	`add_note(subject, text)` stores a note for the user	Must
FR-3	`generate_quiz(subject, count, difficulty)` returns questions built from the user's notes	Must
FR-4	`grade_answer(question_id, answer)` returns correct/incorrect, score, short explanation	Must
FR-5	`schedule_review(subject)` returns topics due today using spaced repetition	Must
FR-6	`list_deadlines()` and `add_deadline(title, due_date)`	Should
FR-7	Voice-friendly output: short sentences, no markdown, under 40 words per spoken reply	Must
FR-8	User authentication and per-user data isolation	Must
FR-9	Web demo chat page that calls the MCP server through an LLM client	Must
FR-10	Graceful errors with spoken-friendly messages	Should

5. Non-Functional Requirements

- **Latency:** tool responses under 3 seconds for notes and reviews; under 8 seconds for quiz generation.
- **Availability:** 99% during judging window; health check endpoint.
- **Security:** HTTPS only, hashed secrets, input validation, rate limiting, no raw secrets in repo.
- **Privacy:** notes visible only to their owner; delete-my-data endpoint.
- **Scalability:** stateless server, so it can scale horizontally.
- **Maintainability:** typed code, linting, 70%+ test coverage on core logic.

6. System Architecture

Flow:

1. Alexa+ (or the web demo client) sends an MCP request over HTTPS.
2. The MCP server authenticates the token and routes to the right tool.
3. The tool reads or writes the database.
4. For quizzes and grading, the tool calls an LLM with the user's notes as context.
5. The server returns a short, voice-friendly result.

Components:

Component	Choice	Why
MCP server	Python 3.11 + official MCP Python SDK + FastAPI	Fast to build, Streamable HTTP support
LLM	Claude API (or any compatible LLM) behind a thin wrapper	Quiz generation and grading
Database	PostgreSQL (prod), SQLite (local/demo)	Simple relational data
ORM	SQLAlchemy + Alembic migrations	Safe schema changes
Cache / rate limit	In-memory first, Redis if time	Keep MVP small
Web demo	React or plain HTML + JS chat page	Shows tool calls live
Hosting	Render or Fly.io free tier, managed HTTPS	Public URL needed
CI	GitHub Actions: lint, test, deploy	Repeatable shipping

Repo layout:

```
studybuddy/
  server/        # MCP server, tools, auth
  server/tools/  # one file per tool
  db/            # models, migrations
  llm/           # prompt templates, LLM wrapper
  web/           # demo chat page
  tests/
  docs/          # PRD, demo script, README
```

7. MCP Tool Contracts

All tools return JSON with a `speech` field (what Alexa says) and a `data` field (structured result).

Tool	Input	Output
`add_note`	`subject: str`, `text: str`	`note_id`, `speech`
`generate_quiz`	`subject: str`, `count: int=5`, `difficulty: easy/medium/hard`	`quiz_id`, `questions[]`, `speech`
`grade_answer`	`question_id: str`, `answer: str`	`correct: bool`, `score: 0-1`, `explanation`, `speech`
`schedule_review`	`subject: str (optional)`	`due_topics[]`, `speech`
`add_deadline`	`title: str`, `due_date: ISO date`	`deadline_id`, `speech`
`list_deadlines`	none	`deadlines[]`, `speech`

8. Authentication and Authorization

- **Production:** OAuth 2.1 with PKCE. The server acts as the resource server and issues short-lived access tokens (15 minutes) plus refresh tokens. Login via email + password or Google sign-in.
- **Token format:** signed JWT with `sub` (user id), `scope`, `exp`. Verify signature and expiry on every MCP request.
- **Scopes:** `notes:read`, `notes:write`, `quiz:use`, `deadlines:manage`.
- **Demo mode:** a single bearer token per demo user, set by environment variable, so judges can test without a full login flow.

- **Passwords:** hashed with Argon2 or bcrypt. Never log tokens.
- **Authorization:** every query filters by `user_id`. No cross-user reads.
- **Check before building:** confirm how Alexa+ expects account linking and auth for a self-hosted MCP server in the track docs, then match it.

9. Database Design

users: id (UUID, PK), email (unique), password_hash, created_at

notes: id (UUID, PK), user_id (FK), subject, text, created_at, updated_at

quizzes: id (UUID, PK), user_id (FK), subject, difficulty, created_at

questions: id (UUID, PK), quiz_id (FK), prompt, expected_answer, source_note_id (FK), type

attempts: id (UUID, PK), user_id (FK), question_id (FK), answer_text, score, correct, created_at

reviews: id (UUID, PK), user_id (FK), subject, ease_factor, interval_days, next_review_at, last_reviewed_at

deadlines: id (UUID, PK), user_id (FK), title, due_date, done (bool)

Indexes: (user_id, subject) on notes and reviews; (user_id, next_review_at) on reviews; (user_id, due_date) on deadlines.

Spaced repetition: simple SM-2 style: a correct attempt grows `interval_days`; a wrong attempt resets it to 1 day.

10. AI and Dataset Plan

Core approach: No model training. The LLM generates questions from the user's own notes (retrieval from the database, then prompt), and grades answers against the expected answer with a rubric prompt.

Why no training: training adds risk and time in a hackathon, and the project is judged on a working demo.

Optional evaluation datasets (use to test quiz and grading quality, or for a small fine-tune if time allows). Verify each link and its license before use:

- SQuAD 2.0 (reading comprehension Q&A): https://huggingface.co/datasets/rajpurkar/squad_v2
- SciQ (science exam questions): https://huggingface.co/datasets/allenai/sciq
- RACE (exam-style reading questions): https://huggingface.co/datasets/ehovy/race

Evaluation plan: take 50 notes from the dataset, generate questions, and have teammates rate them on clarity and correctness (target: 85% acceptable). Compare LLM grading against the dataset answers (target: 90% agreement).

Prompt safety: treat note text as data, not instructions, to reduce prompt injection.

11. Security and Privacy

- Validate all inputs with Pydantic schemas; cap note length (5,000 characters).
- Rate limit: 60 requests per minute per user.
- Secrets only in environment variables; `.env` in `.gitignore`.
- CORS limited to the demo origin.
- Delete-account endpoint wipes all user rows.
- Log request ids and errors, not note contents.

12. Error Handling

- Unknown subject → "I could not find notes on that. Want to add one?"
- LLM failure or timeout → retry once, then a short apology and a suggestion to try again.
- Expired token → 401 with a clear re-login message.

13. Testing Plan

- **Unit:** each tool, SM-2 logic, token verification.
- **Integration:** full flow through the MCP Inspector against a test database.
- **Security:** cross-user access test must fail; invalid and expired token tests.
- **Demo rehearsal:** run the demo script twice end to end on the deployed URL.

14. Team of 4: Roles and Flow

Member	Role	Owns
1	Tech lead and backend	MCP server, tool contracts, auth
2	Data and AI	Database, LLM prompts, quiz and grading quality, evaluation
3	Frontend and integration	Web demo chat, Alexa+ connection testing

Member	Role	Owns
4	DevOps, QA, and demo	Deployment, CI, tests, README, demo video, Devpost write-up

Working rules: one GitHub repo, short feature branches, pull requests reviewed by one other member, 15-minute daily sync, and a shared task board.

15. Milestones

Adjust dates once the submission deadline is confirmed on Devpost.

Phase	Output
Phase 1: Setup	Repo, skeleton MCP server running locally, tool contracts agreed
Phase 2: Core	Database, auth, `add_note` and `generate_quiz` working
Phase 3: Complete	`grade_answer`, `schedule_review`, deadlines, web demo
Phase 4: Harden	Tests, rate limits, deploy to public HTTPS, Alexa+ connection check
Phase 5: Ship	Demo video, README, Devpost submission, final rehearsal

Rule: freeze new features one day before the deadline.

16. Demo Plan (2–3 minutes)

1. Problem in one sentence (10 sec).
2. Voice: "Save a note: deadlock needs four conditions." Show the stored note.
3. Voice: "Quiz me on operating systems." Show generated question.
4. Answer by voice; show score and explanation.
5. "What should I review today?" Show the schedule.
6. Show architecture slide and the MCP tools list (15 sec).
7. Close with impact and next steps.

17. Risks and Mitigations

Risk	Mitigation
Alexa+ access not available in time	Web demo client that talks to the same MCP server
LLM cost or rate limits	Cache quizzes, cap question count, use a smaller model
Poor quiz quality	Prompt iteration plus evaluation set from Section 10
Free hosting sleeps	Keep-alive ping before judging; fallback video
Scope creep	Must-haves first; stretch goals only after Phase 4

18. Submission Checklist

- [ ] Public repo with README and setup steps
- [ ] Deployed HTTPS MCP URL and working demo
- [ ] Demo video under the allowed length
- [ ] Architecture diagram
- [ ] Devpost description with team, tech, and impact
- [ ] Rules and eligibility re-checked on Devpost

19. Open Questions

- Exact Alexa+ account linking and auth requirements for the track.
- Allowed LLM providers and any API credits from the hackathon.
- Final submission deadline and video length limit.